**GMRIT/ADM/F-03**
**REV.: 03**

# COURSE HANDOUT

## B. Tech - 3rd Semester

Course Title : Data Structures Lab Dated: 14-08-2023

Course Code : 21CS307 Academic Year : 2023-2024

Course Structure : 0-0-3

Course Coordinator **: Dr. S. Akila Agnes.**

Instructor(s) **: Dr. V. Vasudha Rani, Dr. Satish M, Dr.V. Srinadh, Dr. S. Akila Agnes, Mrs. M. Sravani, Mr. G. Dharma Raju, Ms. P. Pooja**

**Course Description :**

The Data Structures Laboratory course gives students hands-on experience with basic data structures and algorithms in various programming languages. This lab applies the theoretical concepts from Data Structures and Algorithms to the design, analysis, and implementation of efficient data storage and retrieval systems. Students will explore linear and non-linear data structures like arrays, linked lists, trees, and graphs during lab sessions. They will learn to choose the right data structure for different problems, deepening their understanding of data organisation and algorithmic efficiency. The lab exercises will involve programming data insertion, deletion, searching, sorting, and manipulation. Real-world problem-solving scenarios will help students write functional code from abstract algorithmic ideas. Students develop versatile skills by implementing data structures in multiple programming languages in the course.

**Scope and Objective :**

**The course content enables students to:**

- Understand various data structures and their importance in algorithm design.
- Implement the linear data structures stack and queue using array.
- Implement various forms of linked lists.
- Implement various hashing techniques using appropriate data structures.
- Demonstrate different Binary Tree Traversals.
- Demonstrate Binary Search Tree operations.
- Understand the advantages of height balanced trees.
- Implement graphs using either adjacency matrix or adjacency list.

**Course Outcomes:**

**After undergoing this course, the students will be able to:**

1. Implement stack and queue data structures using array and linked list
2. Demonstrate the applications of stack and queue data structures
3. Implement sorting and searching algorithms and to compare their efficiency
4. Implement binary tree, binary search tree and tree traversals
5. Solve graph problems using appropriate data structure

6. Implement hashing techniques for real world applications (telephone directory, dictionary)

**Textbook (s)**

1. Mark Allen Weiss, Data Structures and Algorithm Analysis in C, Pearson Education, 2002
2. Michael Main, Walter Savitch, Data Structures and other objects using C++, 4th Edition, Addison Wesley, 2018

**Reference (s)**

1. S. Tanenbaum, Y. Langsam and M.J. Augenstein,, Data Structures using C and C++, 2nd Edition, Pearson Education, 2015
2. R. F. Gilberg, B. A. Forouzan, Data Structures A Pseudocode Approach with C, 2nd Edition, CENGAGE Learning, 2005

**CO-PO Mapping:**

| COs | $PO_2$ | $PO_3$ | $PO_4$ | $PO_5$ |
|---|---|---|---|---|
| 1 | 3 | 2 | 3 | 2 |
| 2 | 3 | 2 | 3 | 1 |
| 3 | 3 | 1 | 2 | 2 |
| 4 | 2 | 2 | 2 | 2 |
| 5 | 2 | 2 | 3 | 2 |
| 6 | 2 | 2 | 2 | 2 |

**List of Experiments**

1. **Arrays**

Implement list using an array and perform the following operations on it.

- **Insert(pos, element):** It should insert the given element into the list at the specified position pos. If pos is out of range (less than 0 or greater than the current list size), display an error message.
- **Delete(pos, element):** This function should delete the given element from the list at the specified position pos, if it exists. If pos is out of range or the element is not found at the given position, print an appropriate message.
- **Search(element):** This function should check if the given element is present in the list. If the element is found, return True; otherwise, return False.
- **Running Sum of 1D Array()**: This function return a new array containing the running sum of input array 'num'. The running sum of an array is defined as
  Running Sum[i] = sum(nums[0]…nums[i]).
- **Display():** This function should display the current elements present in the list.

**Examples:**

Initial list: []
Insert(0, 5): [5]
Insert(1, 10): [5, 10]
Insert(5, 15): Invalid position, element not inserted
Display: [5, 10]

Initial list: [1, 2, 3, 4]
Delete(1, 2): [1, 3, 4]
Delete(0, 5): Element not found at position 0

Display: [1, 3, 4]

List: [1, 2, 3, 4]
Search(3): True
Search(5): False

Given array: [1, 2, 3, 4]
Running Sum: [1, 3, 6, 10]

Given array: [2, 4, 6, 8]
Running Sum: [2, 6, 12, 20]

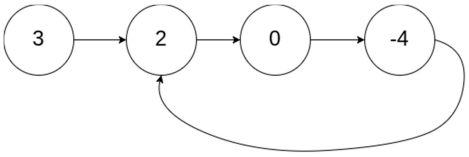
2. **Singly linked list**

Implement a singly linked list with the following functions: insert, delete, display and detect_cycle.

**Node Structure**:
Each node should have two attributes: val to store the value of the node and next to point to the next node in the list.

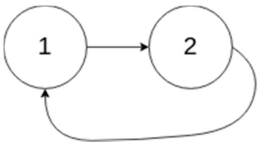
- **insert(val)**: This function should take an integer val as input and insert a new node with the given value at the end of the linked list.
- **delete(val)**: This function should take an integer val as input and remove the first occurrence of the specified value from the linked list. If the value is not found in the list, print an appropriate message.
- **display():** This function should display all the elements present in the linked list, starting from the head to the tail.
- **detect_cycle():** This function should detect if there is a cycle in the linked list. It should return True if a cycle is found and False otherwise. There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer.

**Example 1:** detect_cycle()




Input: head = [3,2,0,-4],
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).

**Example 2**: detect_cycle()



Input: head = [1,2]
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.
**Example 3**: detect_cycle()

Input: head = [1], pos = -1
Output: false
Explanation: There is no cycle in the linked list.

3. **Doubly linked list**

Implement a doubly linked list with the following functions: insert, delete, display, and middle_node.

**Node Structure:**
Each node should have three attributes: val to store the value of the node, prev to point to the previous node, and next to point to the next node in the list.

- **insert(val):** This function should take an integer val as input and insert a new node with the given value at the end of the doubly linked list.
- **delete(val):** This function should take an integer val as input and remove the first occurrence of the specified value from the doubly linked list. If the value is not found in the list, print an appropriate message.
- **display():** This function should display all the elements present in the doubly linked list, starting from the head to the tail.
- **middle_node():** This function should find and return the middle node of the doubly linked list. If the list has an even number of nodes, return the first middle node.

**Example 1**: middle_node(
Input: head = [1,2,3,4,5]
Output: [3,4,5]
Explanation: The middle node of the list is node 3.

**Example 2**: middle_node(
Input: head = [1,2,3,4,5,6]
Output: [4,5,6]
Explanation: Since the list has two middle nodes with values 3 and 4, we return the first one.

4. **Linear search and Binary search**

**Linear search**
You are given an m x n integer matrix with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.
- Given an integer target, return true if target is in matrix or false otherwise.

**Example 1**:

| | | |
|---|---|---|
| 1 | 3 | 5 |
| 10 | 11 | 16 |

Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true

**Binary Search:**
Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.

**Example 1**:

Input: nums = [3,0,1]
Output: 2
Explanation: n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.
**Example 2:**

Input: nums = [0,1]
Output: 2
Explanation: n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.
**Example 3:**

Input: nums = [9,6,4,2,3,5,7,0,1]
Output: 8
Explanation: n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.

**Constraints:**
All the numbers of nums are unique.

5. **Stack**

**Remove All Adjacent Duplicates in String** using stack
Implement a stack Abstract Data Type (ADT) that supports the following operations:

- push(element): Add the given element to the top of the stack.
- pop() -> element: Remove and return the top element from the stack. If the stack is empty, return None.
- peek() -> element: Return the top element of the stack without removing it. If the stack is empty, return None.

Implement a function that removes all adjacent duplicates in the string using a stack-based approach. A duplicate removal operation consists of selecting two adjacent and equal letters and eliminating them from the string. You should repeatedly apply these duplicate removals on the string s until no further duplicates can be removed.

- Input: A string s ($1 \leq |s| \leq 10^5$) consisting of lowercase English letters.
- Output: Return a string representing the final string after all adjacent duplicates have been removed.

**Example 1:**
Input: s = "abbaca"
Output: "ca"
Explanation:
For example, in "abbaca" we could remove "bb" since the letters are adjacent and equal, and this is the only possible move. The result of this move is that the string is "aaca", of which only "aa" is possible, so the final string is "ca".
**Example 2:**
Input: s = "azxxzy"
Output: "ay"

6. **Stack applications: Infix to postfix conversion**

Implement function to convert infix expression to its postfix notation using a stack.

Infix notation is the commonly used notation to represent mathematical expressions, where operators are written between the operands. For example, 2 + 3 * 4 is an infix expression.Postfix notation (also known as Reverse Polish Notation) is a way to represent mathematical expressions where each operator follows its operands. For example, the postfix equivalent of 2 + 3 * 4 is 2 3 4 * +.

- Input:A string infix_expr containing the infix expression. The expression will only contain the following characters: digits (0-9), operators (+, -, *, /), and parentheses (, ).
- Output:A string representing the postfix expression obtained after converting the infix expression.

**Example 1:**
Input: infix_expr = "2 + 3 * 4"
Output: "2 3 4 * +"

7. **Stack applications: Evaluation of postfix expressions.**
Postfix expression as input and evaluates it to obtain the result using a stack.
- Input: A string postfix_expr containing the postfix expression. The expression will only contain the following characters: digits (0-9) and operators (+, -, *, /).
- Output: An integer representing the result of the evaluated postfix expression.

**Examples:**
Input: postfix_expr = "2 3 4 * +"
Output: 14
Explanation: The postfix expression is evaluated as 2 + (3 * 4) = 2 + 12 = 14.

8. **Queue**

Implement the queue Abstract Data Type (ADT) and use it to calculate the time needed to buy tickets.
Problem Statement: **Time Needed to Buy Tickets**

- There are n people in a line queuing to buy tickets, where the 0th person is at the front of the line and the (n - 1)th person is at the back of the line.
- You are given a 0-indexed integer array tickets of length n where the number of tickets that the ith person would like to buy is tickets[i].
- Each person takes exactly 1 second to buy a ticket. A person can only buy 1 ticket at a

time and has to go back to the end of the line (which happens instantaneously) in order to buy more tickets. If a person does not have any tickets left to buy, the person will leave the line.

- Return the time taken for the person at position k (0-indexed) to finish buying tickets.

**Example 1:**
Input: tickets = [2,3,2], k = 2
Output: 6
Explanation:
- In the first pass, everyone in the line buys a ticket and the line becomes [1, 2, 1].
- In the second pass, everyone in the line buys a ticket and the line becomes [0, 1, 0].
The person at position 2 has successfully bought 2 tickets and it took 3 + 3 = 6 seconds.

**Example 2:**
Input: tickets = [5,1,1,1], k = 0
Output: 8
Explanation:
- In the first pass, everyone in the line buys a ticket and the line becomes [4, 0, 0, 0].
- In the next 4 passes, only the person in position 0 is buying tickets.
The person at position 0 has successfully bought 5 tickets and it took 4 + 1 + 1 + 1 + 1 = 8 seconds.

9. **Insertion sort**

Sort the array using insertion sort, and then find the **majority element** is the element that appears more than ⌊n / 2⌋ times. You may assume that the majority element always exists in the array.

**Example 1:**
Input: nums = [3,2,3]
Output: 3
**Example 2:**
Input: nums = [2,2,1,1,1,2,2]
Output: 2

10. **Merge sort**

Given an array of integers nums, sort the array in ascending order and return it.
Solve the problem without using any built-in functions in O(nlog(n)) time complexity and with the smallest space complexity possible.

**Example 1:**
Input: nums = [5,2,3,1]
Output: [1,2,3,5]
Explanation: After sorting the array, the positions of some numbers are not changed (for example, 2 and 3), while the positions of other numbers are changed (for example, 1 and 5).
**Example 2:**
Input: nums = [5,1,1,2,0,0]
Output: [0,0,1,1,2,5]
Explanation: Note that the values of nums are not necessairly unique.

11. **Hash Table**

Find the first non-repeating character in a string using hash table,Given a string s, find the first non-repeating character in it and return its index. If it does not exist, return -1.

Example 1:
Input: s = "leetcode"
Output: 0
Example 2:
Input: s = "loveleetcode"
Output: 2
Example 3:
Input: s = "aabb"
Output: -1

12. **Open hashing**

Implement the Open Hashing data structure that supports the following operations:

- **insert(key, value)**: Insert a key-value pair into the hash table.
- **get(key)** -> value: Retrieve the value associated with a given key from the hash table.
- **remove(key)**: Remove the key-value pair with the given key from the hash table.

**Constraints:**

- Use an appropriate hash function to convert keys into valid indices for the hash table.
- If a collision occurs during insertion (i.e., multiple keys map to the same index), use separate chaining to handle it by creating a linked list at that index.

13. **Tree traversal algorithms**

You are given a binary tree. Your task is to implement three tree traversal algorithms: Preorder Traversal, Inorder Traversal, and Postorder Traversal.

- Preorder Traversal: Visit the root node first, then traverse the left subtree, and finally traverse the right subtree.
- Inorder Traversal: Traverse the left subtree first, then visit the root node, and finally traverse the right subtree.
- Postorder Traversal: Traverse the left subtree first, then traverse the right subtree, and finally visit the root node.

**Example :**
Input: root = [1,2,3,4,5,null,null]
Output:
Preorder: [1, 2, 4, 5, 3]
Inorder: [4, 2, 5, 1, 3]
Postorder: [4, 5, 2, 3, 1]

14. **Binary Search Tree (BST)**

The Binary Search Tree must maintain its BST property at all times. For every node in the BST, the keys in its left subtree are less than its key, and the keys in its right subtree are greater than its key. Implement the Binary Search Tree data structure that supports the following operations:

- **insert(key):** Insert a new key into the BST while maintaining its BST property.
- **delete(key):** Delete a key from the BST while preserving its BST property.
- **search(key)** -> bool: Check whether a given key is present in the BST.

Example:
Insertion of elements (50, 30, 20, 40, 70, 60, 80) into the BST.

```
        50
       /  \
     30    70
    / \   / \
  20  40 60  80
```

Constraints:
The BST property is maintained throughout the insertions and deletions.

15. **Balanced Binary Tree**

Given a binary tree, determine if it is height-balanced. A binary tree is height-balanced if the difference in heights between its left and right subtrees is at most 1 for every node in the tree.Write a function called is_balanced that takes the root of the binary tree as input and returns a boolean value indicating whether the binary tree is height-balanced or not.

**Example 1:**
Input: root = [3,9,20,null,null,15,7]
Output: true
**Example 2:**
Input: root = [1,2,2,3,3,null,null,4,4]
Output: false
**Example 3:**
Input: root = []
Output: true

16. **Graph traversal algorithm**

Implement the Depth First Search (DFS) algorithm to traverse the graph starting from a given source vertex. It that takes the adjacency matrix representation of a graph and the source vertex as inputs and returns a list of vertices visited during the DFS traversal.
**Example**
adj_matrix = [ [0, 1, 0, 1, 0], [1, 0, 1, 0, 1], [0, 1, 0, 1, 0], [1, 0, 1, 0, 1], [0, 1, 0, 1, 0]]
source = 2
DFS traversal order is [2, 1, 0, 3, 4]
Source =0
DFS traversal order is [0, 1, 2, 3, 4]

**Indicative list of applications-based experiments**

1. Given an expression exp of length n consisting of some brackets, the task is to print the bracket numbers when the expression is being parsed.
   **Input:**
   The first line contains an integer **T,** the number of test cases. For each test case, there is a string exp containing the expression.
   **Output:**
   For each test case, the output is the bracket numbers of the expression.
2. Given an unsorted array **arr[]** of size **N**, rotate it by **D** elements(clockwise).
   **Input:**
   The first line of the input contains **T** denoting the number of test cases. First line of each test case contains two space separated elements, **N** denoting the size of the array and an

integer **D** denoting the number size of the rotation. Subsequent line will be the N space separated array elements.

**Output:**

For each test case, in a new line, output the rotated array.

**Example:**

**Input:**

2
5 2
1 2 3 4 5
10 3
2 4 6 8 10 12 14 16 18 20

**Output:**

3 4 5 1 2
8 10 12 14 16 18 20 2 4 6

3. Given a singly linked list, the task is to find the length of the linked list, where the length is defined as the number of nodes in the linked list.

   **Input:**

   FirstlineofinputcontainsnumberoftestcasesT.Toreachtestcase,firstlineofinputcontainsnumber of nodes N, to be inserted into the linked list and next line contains data of Nnodes.

   **Output:**

   There will be a single line of output for each test case, which contains length of the linked list.

4. Given a doubly linked list, rotate the linked list counter-clockwise by P nodes. Here, P is a given positive integer and is smaller than the count of nodes (N) in the linked list.

   **Input:**

   The first line of input contains an integer **T** denoting the no of test cases. For each test case, the first line of input contains two integers **N and P** denoting the number of nodes in Linked List and the number of nodes to be rotated respectively.

   **Output:**

   For each test case, output the final linked list after the P rotations in it**.**

5. Givenastackwith**push(),pop(),empty()**operations,delete**middle**ofitwithoutusinganyadditional datastructure.

   **Middle:** ceil(size_of_stack/2.0)

   **Input Format:**

   The first line contains an integer **T**, the number of test cases. For each test case, the first line contains aninteger**sizeOfStack**denotingthestacksize.Nextlinecontainsspaceseparatedintegersthatwillbe pushed into thestack.

   **Output Format:**

   For each test case, in a new line, print the stack elements. **If stack size if 1 then just print**

6. Given a Queue **Q** containing **N** elements. The task is to reverse the Queue. Your task is to complete the function **rev(),** that reverses the **N** elements of thequeue.

   **Input Format:**

   The first line of input contains an integer **T** denoting the Test cases. Then **T** test cases follow. The first line contains N which is the number of elements which will be reversed. Second line contains N space separated elements.

   **Output Format:**

   For each test case, in a new line, print the reversed queue.

7. Given a Binary Search Tree, find the sum of all leaf nodes. BST has the following property (duplicate nodesarepossible):The**leftsubtree**ofanodecontainsonlynodeswith**keysless**thanthenode'skey; The **right sub tree** of a node contains only nodes with **keys greater than** or **equal** to the node's key. **Input:**

   The first line of input contains a single integer **T** denoting the number of test cases. Then T test cases follow.EachtestcaseconsistsoftwolinesThefirstlineofeachtestcaseconsistsofinteger**N**,denoting

the number of elements in the BST. The second line of each test case consists of N space-separated integers denoting the elements in the BST.

**Output:**

For each test case, in a new line, print the sum of leaf nodes.

**Indicative list of Augmented Experiments:**

1. Implementation of symbol table
2. Evaluation of expression by constructing expression tree
3. Implementation of hash table with double hashing
4. Implementation of dictionary using hashing technique
5. Implementation of graph algorithms

**Reading Material (s)**


1. Data Structures Lab manual, Department of CSE & IT, GMRIT, Rajam
2. https://leetcode.com/

**Course Plan:**

| Lab Session No. | Learning Objectives: | Topics to be covered |
|---|---|---|
| 1 | To understand the list operations. | Arrays Operations: Insert, Delete, Search, Running Sum, Display |
| 2 | To understand the Linked list operations in one way traversal | Singly Linked List: Insert, Delete, Display, Detect Cycle |
| 3 | To understand the Linked list operations in two-way traversal | Doubly Linked List: Insert, Delete, Display, Find Middle Node |
| 4 | To learn and understand the difference between linear and binary search algorithms. | Linear Search and Binary Search |
| 5 | To implement a stack and perform basic stack operations and remove adjacent duplicates from a given sequence using a stack. | Stack ADT and Adjacent Duplicates Removal |
| 6 | To implement the algorithm to convert infix expressions to postfix using a stack. | Stack Application: Infix to Postfix Conversion |
| 7 | To implement the stack-based algorithm for postfix expression evaluation. | Stack Application: Postfix Expression Evaluation |
| 8 | To Understand and Implement a queue data structure and apply queues to model real-world scenarios. | Queue ADT and Time Needed to Buy Tickets |
| 9 | To implement insertion sort to arrange elements in a given sequence. | Insertion Sort and Majority Element Detection |
| 10 | To understand the divide-and-conquer strategy and implement the merge sort algorithm to sort a list of elements. | Merge Sort Implementation |
| 11 | To implement a hash table and its associated collision resolution method. | Hash Table: First Non-Repeating Character |
| 12 | To implement open hashing to handle collisions in a hash table. | Open Hashing Implementation |
| 13 | To implement recursive algorithms to traverse binary trees in various orders. | Tree Traversal Algorithms: Preorder, Inorder, Postorder |
| 14 | To implement basic BST operations: insertion, deletion, and searching. | Binary Search Tree (BST) Operations |
| 15 | To implement an algorithm to determine if a binary tree is balanced. | Balanced Binary Tree Detection |
| 16 | To implement DFS to traverse graphs | Graph Traversal: Depth First Search (DFS) |

**Evaluation scheme:**

| Component | Particular | Marks | Date & Time |
|---|---|---|---|
| Day to Day Performance | No of Experiments completed and recorded | 15 | 14-08-2023 to 28-11-2023 |
| Internal Examination + Augmented Experiment | 150 minutes | 5 + 5 | 29-11-2023 to 06-12-2023 |
| External Examination | 180 minutes | 50 | 18-12-2023 to 23-12-2023 |
| | Total | 75 | |

**Chamber Consultation Hour:** 4.00PM

**Venue:** CSE / IT Staff Room

**Notices:** CSE / IT Notice Board

**Course Coordinator**

**Dr. S. Akila Agnes**, Sr.Asst. Prof/ CSE

**Course Instructors**

**Dr. V. Vasudha Rani**, Assoc. Prof/ IT

**Dr. V. Srinadh**, Assoc. Prof / CSE-AIML

**Dr. Satish M**, Assoc. Prof / CSE

**Dr. S. Akila Agnes**, Sr. Asst. Prof/ CSE

**Mr. G. Dharma Raju**, Asst. Prof/ CSE

**Mrs. M. Sravani**, Asst. Prof/ CSE

**Ms. P.Pooja**, Asst. Prof/ IT

**HOD/CSE**